GIDFINE — PHASE 3A IMPLEMENTATION AUDIT & INTEGRATION REPORT

Branch: development | **Stack:** React 19 + TypeScript + Convex | **Spec:** Gidfine_Dev_Guide.md v1.1

1. Executive Summary

Phase 3A integrates the existing React frontend with the secure Phase 1 (schema & state machine) and Phase 2 (mutations & server-side authorization) Convex backend. The implementation establishes a typed client service adapter, server-side borrower/agency queries, borrower application tracking and affirmative Act 843 consent signing, and an agency review queue dashboard with disbursement audit capabilities. All 24 Node unit/integration tests and 6 Python regression tests pass cleanly with 0 build errors.

2. Verification & Safety Controls

Requirement	Status	Audit Details
Git Branch Safety	**PASSED**	Checked: strictly on branch *development*. Never touched *main*.
Convex Cloud Deployment	**PASSED**	Zero cloud provisioning executed. Awaits Matthias configuration.
Python Backend Integrity	**PASSED**	All FastAPI routes in api/ and tests/ preserved unchanged.
Authentication Guard	**PASSED**	Identity resolved exclusively from ctx.auth.getUserIdentity(). No mocks.
Financial Precision	**PASSED**	All currency in integer pesewas, rates in basis points. No invented formulas.
Cloud Unavailability Handling	**PASSED**	Explicit ConvexCloudUnavailableError thrown. Never returns false success.

3. Server-Side Queries (frontend/convex/loans.ts)

• **getBorrowerApplications:** Enforces borrower role check via authenticated identity. Constrains query results strictly to borrowerId == user._id and agencyId == user.agencyId. Automatically redacts proposals in drafting/review states (DRAFTED, LENDER_SUBMITTED, LENDER_ACCEPTED), only exposing terms once AGENCY_APPROVED.
• **getAgencyReviewQueue:** Enforces agency roles (agency_admin, loan_officer, underwriter, cashier, auditor). Derives agencyId server-side from verified caller record rather than client input. Redacts borrower PII and supports status filtering.

4. Typed Service Adapter (frontend/src/services/loanService.ts)

• **Currency & Rate Helpers:** Deterministic ghsToPesewas (Math.round), pesewasToGhs, formatted pesewasToGhsString (GH■ X,XXX.XX), and bpsToPercentString.
• **Explicit Cloud Failure:** Raises ConvexCloudUnavailableError if invoked without a connected Convex client (VITE_CONVEX_URL unconfigured).
• **Client-side Guards:** Asserts minimum principal (GH■ 100) and affirmative consent (consentGiven === true) before dispatching to backend.

5. UI Components Integrated

• **BorrowerLoanManager (LoanApplication.tsx):** Direct Convex Workflow toggle, application request form with integer pesewas conversion, real-time application tracking cards, and Act 843 affirmative consent signing modal with SHA-256 fingerprinting.
• **AgencyReviewQueue (AdminDashboard.tsx):** Multi-status review queue table, terms proposal modal, agency approval button, and disbursement readiness audit modal inspecting consent fingerprint and funding lockout.

6. Verification & Test Summary

Test Suite	Passed	Failed	Coverage Areas
State Machine Tests	7	0	Linear transitions, assigned lender checks, agency & borrower guards, SHA-256 fingerprint.
Convex Mutation & Query Tests	11	0	Unauthenticated rejection, role authorization, tenant isolation, PII redaction, query isolation.
LoanService Adapter Tests	6	0	Pesewas conversions, bps formatting, cloud error throwing, input validations.
Python Regression (pytest)	6	0	All FastAPI loan engine endpoints preserved.
TypeScript & Production Build	1845 mods	0 errors	tsc -b and vite build clean in 9.22s.

7. Remaining Dependencies

1. Cloud Deployment: Matthias to run convex dev/deploy and provide VITE_CONVEX_URL.
2. Auth Provider: Production JWT issuer configuration in frontend/convex/auth.config.ts.
3. Phase 3B Scope: Lender portal for reviewing assigned quotes, submitting proposals, and executing disbursement webhooks.